COMPANION TO THE RAC ON KUBEADM LAB GUIDE

How the ASM disks reach the pods

Follow one shared disk through nine layers, from a VirtualBox file on Windows to a member of the ASM disk group inside two RAC pods: what each layer names it, who creates that name, why it exists, and how to see it.

`VirtualBox shareable VDI` `udev /dev/disk/by-id` `Block PV and PVC` `volumeDevices` `CRI-O` `Oracle ASM`

25 Oct 2026 · The lab follows Oracle's How to Install and Configure Oracle RAC on Oracle Cloud Native Environment. This page explains guide steps 3.3, 7, 8.2, 10.4 and 11.3 from the disk's point of view.

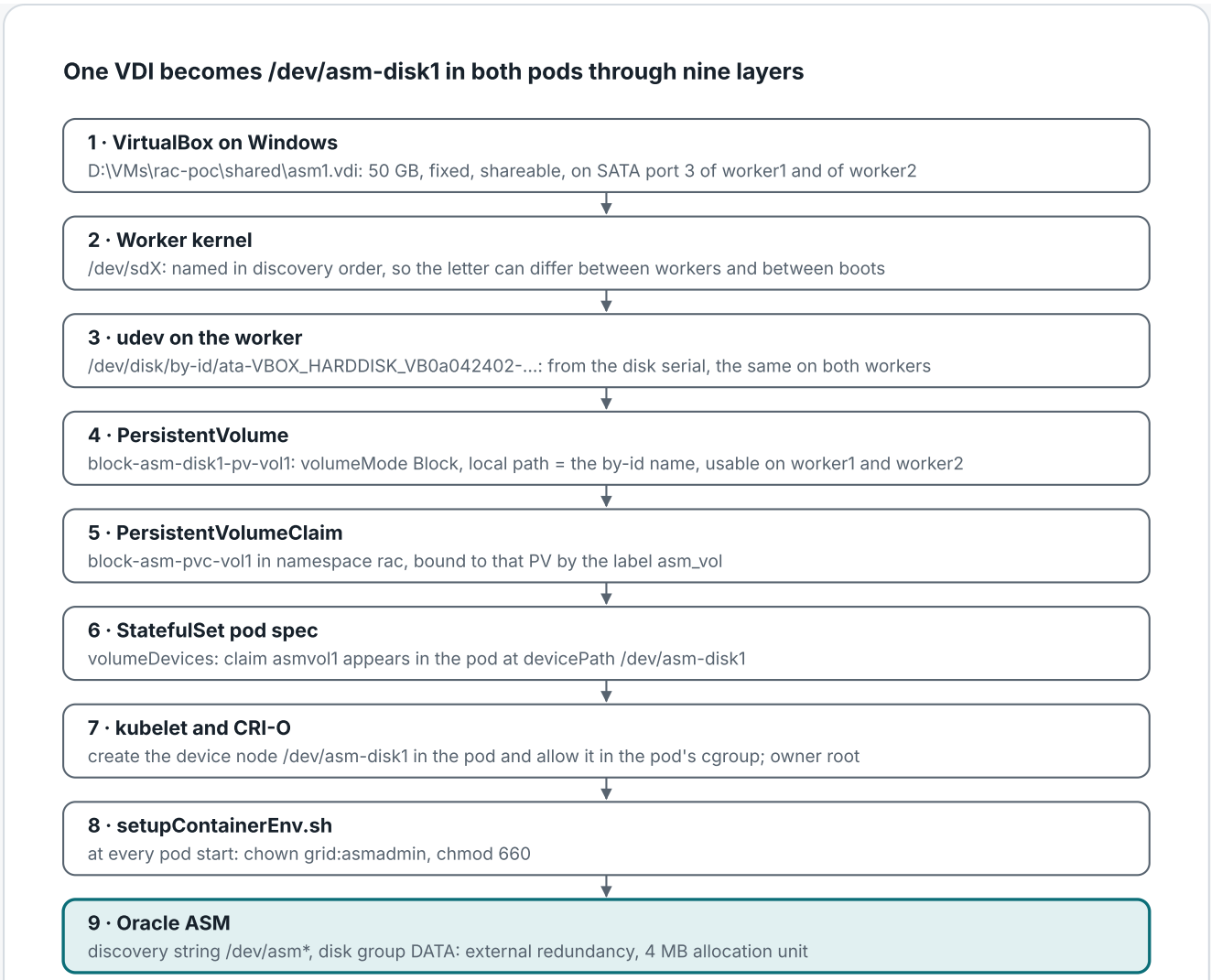
`PowerShell` on your PC

`worker · controlplane` root shell on a VM

`racnode1-0` root shell in a RAC pod

The chain at a glance

RAC needs every instance to read and write the same disks. In this lab that means two 50 GB virtual disks that both RAC pods open as `/dev/asm-disk1` and `/dev/asm-disk2`. Between the VirtualBox file on Windows and the device ASM opens inside a pod, the disk passes through nine layers. Each layer gives it a different name, and each one exists for a reason.



ASM disk 1 from the VDI file to ASM. The same chain runs on each worker and ends at the same file.

The same chain runs twice in parallel, once on each worker, and both chains end at the same file. That's what makes the disk shared.

#	LAYER	DISK 1 IS CALLED	CREATED BY	GUIDE STEP
1	VirtualBox	`D:\VMs\rac-poc\shared\asm1.vdi`	`VBoxManage createmedium ... --variant Fixed`, `--type shareable`	3.3
2	Worker kernel	`/dev/sdd` or similar; the letter isn't fixed	the kernel's SATA driver, in discovery order	automatic
3	udev on the worker	`/dev/disk/by-id/ata-VBOX_HARDDISK_VB0a042402-…`	udev's default rules, from the disk serial	automatic
4	PersistentVolume	`block-asm-disk1-pv-vol1`	`kubectl apply`	7.3
5	PersistentVolumeClaim	`rac/block-asm-pvc-vol1`	`kubectl apply`	7.3

#	LAYER	DISK 1 IS CALLED	CREATED BY	GUIDE STEP
6	Pod spec	volume `asmvol1`, `devicePath: /dev/asm-disk1`	the StatefulSets	10.4
7	Device node in the pod	`/dev/asm-disk1`, owner `root`	kubelet and CRI-O, at every container start	automatic
8	Ownership in the pod	`/dev/asm-disk1`, `grid:asmadmin 660`	`setupContainerEnv.sh` from `rc.local`	8.2 (image)
9	Oracle ASM	disk `DATA_0000` in disk group `DATA`	`gridSetup.sh` and its response file	11.3, 11.4

The two most useful names to remember: `/dev/disk/by-id/...` is how the workers find the disk (layer 3), and `/dev/asm-disk1` is how the pods and ASM know it (layers 6 to 9). Everything between them is Kubernetes translating one into the other.

Layer 1: VirtualBox, one file attached to two VMs

What: each ASM disk is one VDI file in `D:\VMs\rac-poc\shared`, attached to worker1 **and** worker2 on the same SATA port (asm1 on port 3, asm2 on port 4).

Why it's built this way:

- `--type shareable`: by default VirtualBox lets one running VM at a time use a disk, and caches writes per VM. Shareable mode lets two VMs use it at once and sends every write straight to the file, so each worker sees the other's writes. That stands in for a SAN LUN zoned to two servers.
- `--variant Fixed`: VirtualBox only allows the shareable type on fixed-size disks, so the full 50 GB is allocated on the host at creation.
- `--mtype shareable` **on** `storageattach`: the attachment must also be marked shareable, on both VMs.
- **No snapshots of the workers:** VirtualBox can't snapshot a VM that has a shareable disk attached.

The serial number matters later. VirtualBox reports each VDI to the guest with the model `VBOX HARDDISK` and a serial built from the disk's UUID: `VB`, the first 8 hex characters of the UUID, a dash, then 8 more characters derived from the end of the UUID. For example, a disk with UUID `0a042402-aaab-4af2-af32-72c3c052dac5` gets a serial starting with `VB0a042402-`. The UUID is new every time a disk is created, so your serials differ from the examples here. Because both workers attach the same file, they both see the same serial.

PowerShell (see layer 1)

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
foreach ($d in "asm1","asm2") { & $VB showmediuminfo disk "D:\VMs\rac-poc\shared\$d.vdi" | Select-String
'^(UUID|Type|Format variant|In use by|  )' }
```

What you should see: for each disk, `Type: shareable`, `Format variant: fixed default`, and both `worker1` and `worker2` under "In use by VMs".

Layer 2 and 3: the worker's kernel and udev

Layer 2, the kernel name (`/dev/sdX`): when a worker boots, the kernel's SATA (AHCI) driver finds four disks besides the OS disk: the 80 GB and 100 GB data disks and the two 50 GB ASM disks. It names them `sdb`, `sdc`, `sdd`, `sde` in the order they're discovered. **That order isn't the VirtualBox port order and isn't guaranteed to match between workers or between boots.** In the first build, `/dev/sdc` on worker2 was an ASM disk, while the setup script expected the 100 GB data disk there. Formatting by name would have wiped the shared disk (issue #7).

Layer 3, the stable name (`/dev/disk/by-id/...`): udev runs on every worker, and its **default** rules (`60-persistent-storage.rules`, shipped with the OS) read each disk's model and serial and create a symbolic link from them:

```text
/dev/disk/by-id/ata-VBOX_HARDDISK_VB0a042402-…  ->  ../../sdd
```

The link name comes from the disk itself, not from discovery order, so it's the same on both workers and after every reboot. Only the target (`sdd`) can differ. **This is the only place udev takes part, and nobody writes a rule for it.**

Why by-id and not the doc's `/dev/sdd`: Oracle's document points the PersistentVolumes at `/dev/sdd` and `/dev/sde`. That's safe on hardware where the names are predictable, but not here. Using `/dev/disk/by-id/...` is a deliberate deviation from the doc.

`worker1, then worker2` (see layers 2 and 3)

CONNECT FIRST `ssh -p 2211 root@127.0.0.1` for worker1, `ssh -p 2212 root@127.0.0.1` for worker2

```
lsblk -d -o NAME,SIZE,SERIAL,TYPE
ls -l /dev/disk/by-id/ | grep VBOX_HARDDISK | grep -v part
blkid /dev/disk/by-id/ata-VBOX_HARDDISK_VB* 2>/dev/null
```

What you should see: three 50 GB disks on each worker, because the OS disk is also 50 GB. The two shared ones are the two whose serial and by-id name appear on **both** workers; the OS disk's serial is different on each worker, because each worker has its own copy. The shared disks can have different `sdX` letters on the two workers (in one rebuild they were `sdc` / `sdd` on worker1 and `sdd` / `sde` on worker2). `blkid` also lists the OS disk and the two data disks (`xfs`). The two ASM disks are missing from it before the Grid install, because they're blank; after it they show `TYPE="oracleasm"`, which is ASM's own header.

Layers 4 and 5: PersistentVolume and PersistentVolumeClaim

Kubernetes doesn't let a pod name a host device directly. A pod asks for a **claim**; the claim is bound to a **volume**; the volume says which device on which nodes. That indirection is what lets the same pod definition run on either worker.

Layer 4, the PersistentVolume (`block-asm-disk1-pv-vol1` **)**, from guide 7.3:

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: block-asm-disk1-pv-vol1
  labels: { asm_vol: block-asm-pv-vol1 }
spec:
  volumeMode: Block
  capacity: { storage: 50Gi }
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  local: { path: /dev/disk/by-id/ata-VBOX_HARDDISK_VB0a042402-… }
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: kubernetes.io/hostname, operator: In, values: [worker1, worker2] }
```

- `volumeMode: Block` **:** the pod gets a raw device, not a mounted filesystem. ASM manages the disk itself and must not have a filesystem on it.
- `local: { path: ... }` **:** a "local" volume is a disk that's physically on the node, with no storage driver in between. The path is the layer 3 name, so it means the right disk on both workers.
- `nodeAffinity` **with both workers:** a local volume only exists where its device exists. Listing both workers tells the scheduler the PV is usable on either. Normally a local PV lists one node; here two nodes really do have the same disk, because of layer 1.
- `ReadWriteMany` **:** both RAC pods use the disk at the same time. Kubernetes doesn't coordinate those writes; ASM and the RAC cluster do, exactly as with a SAN.
- `Retain` **:** deleting the claim never touches the disk. Nothing deletes or wipes it automatically.
- **No StorageClass:** these volumes are created by hand ("static provisioning"), so there's no driver that could create, resize or delete disks.

Layer 5, the PersistentVolumeClaim (`rac/block-asm-pvc-vol1` **):**

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: block-asm-pvc-vol1
  namespace: rac
spec:
  accessModes: [ReadWriteMany]
  volumeMode: Block
  storageClassName: ""
  resources: { requests: { storage: 50Gi } }
  selector: { matchLabels: { asm_vol: block-asm-pv-vol1 } }
```

- `selector` **on the label** `asm_vol` **:** without it Kubernetes could bind claim 1 to either 50 GB volume. The label pins claim 1 to disk 1 and claim 2 to disk 2.
- `storageClassName: ""` **:** "don't create a volume for me, bind to an existing one".
- **The claim is the only thing the pod refers to.** It lives in namespace `rac` ; the PV has no namespace.

`controlplane` (see layers 4 and 5)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl get pv -o custom-
columns=PV:.metadata.name,MODE:.spec.volumeMode,PATH:.spec.local.path,CLAIM:.spec.claimRef.name,STATUS:.s
tatus.phase
kubectl -n rac get pvc
```

What you should see: both PVs `Bound` , each to its own claim, each with the by-id path of its own disk.

Layers 6 and 7: the pod spec, kubelet and CRI-O

Layer 6, the pod spec. Each RAC StatefulSet (guide 10.4) names the claims and says where the devices should appear inside the container:

```yaml
    volumes:
      - name: asmvol1
        persistentVolumeClaim: { claimName: block-asm-pvc-vol1 }
      - name: asmvol2
        persistentVolumeClaim: { claimName: block-asm-pvc-vol2 }
    containers:
      - name: racnode1-0
        volumeDevices:
        - { name: asmvol1, devicePath: /dev/asm-disk1 }
        - { name: asmvol2, devicePath: /dev/asm-disk2 }
```

- `volumeDevices`, **not** `volumeMounts`: a block volume is exposed as a device file; there's nothing to mount.
- `devicePath`: the name inside the container. This is where `/dev/asm-disk1` comes from: it's chosen here, the same in both pods, whatever the disk is called on the worker. It does the job a udev alias such as `/dev/oracleasm/disk1` does on bare metal.
- **Both StatefulSets use the same two claims.** That's allowed because the claims are `ReadWriteMany`.
- **The pod is not privileged** (`privileged: false`). It can reach exactly the devices listed here and no others.

Layer 7, the device node. When the pod is scheduled on a worker, that worker's kubelet:

1. finds the PV behind each claim and checks the PV's `nodeAffinity` allows this node;
2. resolves the local path `/dev/disk/by-id/...` to the real device on this node (for example `/dev/sdd`);
3. records the mapping under `/var/lib/kubelet` (a "global map path" for the volume and a per-pod link to it);
4. passes the container runtime (CRI-O) a device entry: host device → container path `/dev/asm-disk1`, with read, write and mknod permission.

CRI-O, through the low-level runtime, then:

- **creates a device node** `/dev/asm-disk1` in the container's own `/dev`, with the same major and minor numbers as the host device, so it's the same disk;
- **adds a cgroup device rule** that allows the container to open that one device. Without the rule, even root in the container would get "operation not permitted".

Two consequences matter for RAC:

- **The node is created as** `root`**, not** `grid`**.** It's a new file in the container's `/dev`, so ownership set on the worker's `/dev/sdd` doesn't carry over. That's why layer 8 exists.
- **It's created again at every container start.** A recreated pod gets fresh, root-owned nodes, so layer 8 must run at every start, not once.

`worker1` (see layers 6 and 7, while racnode1-0 runs there)

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
ls -l /dev/disk/by-id/ | grep VBOX_HARDDISK | grep -v part
ls -l /dev/sd?
find /var/lib/kubelet -path '*volumeDevices*' -name 'block-asm*' 2>/dev/null
```

`racnode1-0` (the same disk, from inside the pod)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`, then `kubectl -n rac exec -it racnode1-0 -- bash`

```
ls -l /dev/asm-disk1 /dev/asm-disk2
```

What you should see: the major and minor numbers of `/dev/asm-disk1` in the pod (the two numbers before the date, such as `8, 48`) match the numbers `ls -l /dev/sd?` shows on the worker for the `sdX` its by-id link points to. The `find` lists the kubelet's map entries for the two PVs.

Layer 8: ownership inside the pod

ASM runs as `grid`, in group `asmadmin`, and must open the disks for reading and writing. On bare metal a udev rule sets that. Inside the pod there's no udev running (the pod runs systemd, but `/dev` is created by the container runtime and isn't managed by udev), so the image does it with a script.

`setupContainerEnv.sh`, built into the RAC image in guide 8.2 and run from `rc.local` at every pod start:

```bash
for d in /dev/asm-disk*; do
  [ -b "$d" ] || continue
  chown grid:asmadmin "$d"
  chmod 660 "$d"
done
```

- `-b`: only real block devices are touched.
- **The fixed IDs matter:** `grid` is UID 54332 and `asmadmin` GID 54334 in the image. The numbers are what's stored, and the Oracle homes on `/u01` (which outlive the pod) use the same IDs.
- **Order at start-up:** in a recreated pod, the restart hook that starts Grid (guide Phase 14) is called at the end of this same script, so the devices already belong to `grid` when ASM starts.

`controlplane` (see layer 8 in both pods)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do kubectl -n rac exec $p -- stat -c "$p %n %U:%G %a" /dev/asm-disk1
/dev/asm-disk2; done
```

What you should see: four lines, all `grid:asmadmin 660`. This is part of the guide's check in 10.5.

Layer 9: Oracle ASM

The Grid response file (guide 11.3) only ever uses the pod-side names:

```text
oracle.install.asm.diskGroup.name=DATA
oracle.install.asm.diskGroup.redundancy=EXTERNAL
oracle.install.asm.diskGroup.AUSize=4
oracle.install.asm.diskGroup.disks=/dev/asm-disk1,/dev/asm-disk2
oracle.install.asm.diskGroup.diskDiscoveryString=/dev/asm*
```

- `diskDiscoveryString=/dev/asm*`: where ASM looks for candidate disks, in every instance. Both pods have the same two names, so both ASM instances find the same disks.
- `EXTERNAL` **redundancy:** ASM doesn't mirror. The two disks are striped into one 100 GB disk group, and protection is left to the storage below (here, none; in production, the SAN's RAID).

- **4 MB allocation unit:** the unit ASM allocates space in.
- **ASM writes its own header to each disk.** That header names the disk group and the disk (`DATA_0000` , `DATA_0001`). It's how ASM recognises its disks whatever their path, and it's what `blkid` on the worker reports as `oracleasm` . It's also why the old ASM disks have to be deleted before a rebuild: a new install refuses disks that already belong to a disk group.

How the instances share the disks safely: both ASM instances, and both database instances, write to the same blocks. Kubernetes and VirtualBox don't prevent conflicts; the RAC cluster does, with its own locking over the private interconnect (Phase 6). This is exactly how RAC uses a SAN.

`racnode1-0` (see layer 9; as grid)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - grid -c "asmcmd lsdsk -k"
su - grid -c "asmcmd lsdg"
```

What you should see: two disks, `/dev/asm-disk1` and `/dev/asm-disk2` , in disk group `DATA` , which is `MOUNTED` with external redundancy (`EXTERN`).

Follow one disk end to end

Run these in order to trace asm1 from the VDI file to ASM. The first block runs on the PC; it opens SSH sessions itself.

`PowerShell` (layers 1 to 3)

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
$pre = ((& $VB showmediuminfo disk "D:\VMs\rac-poc\shared\asm1.vdi") | Select-String '^UUID:\s+([0-9a-f]
{8})').Matches[0].Groups[1].Value
"layer 1: asm1.vdi UUID starts with $pre"
foreach ($p in 2211, 2212) { ssh -p $p root@127.0.0.1 "hostname; ls -l /dev/disk/by-id/ | grep -v part |
grep VB$pre-; stat -L -c 'major:minor %t:%T' /dev/disk/by-id/ata-VBOX_HARDDISK_VB$pre-*" }
```

controlplane (layers 4 to 9)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
echo "layer 4  $(kubectl get pv block-asm-disk1-pv-vol1 -o jsonpath='{.spec.local.path}
{.status.phase}')"
echo "layer 5  $(kubectl -n rac get pvc block-asm-pvc-vol1 -o jsonpath='{.spec.volumeName}
{.status.phase}')"
echo "layer 6  $(kubectl -n rac get sts racnode1 -o
jsonpath='{.spec.template.spec.containers[0].volumeDevices[0]}')"
for p in racnode1-0 racnode2-0; do
  echo "layers 7-8 in $p: $(kubectl -n rac exec $p -- stat -c '%n major:minor %t:%T %U:%G %a' /dev/asm-
disk1)"
done
echo "layer 9  $(kubectl -n rac exec racnode1-0 -- su - grid -c 'asmcmd lsdsk -k' | grep asm-disk1)"
```

What you should see:

- layer 1: the first 8 characters of asm1's UUID (it changes every time the disks are created);
- layers 2-3: for each worker, its name, then the **same** by-id name pointing to some `../../sdX` (the letter may differ), and the device's major:minor in hex (such as `8:30`);
- layer 4: the PV's path is that by-id name, and the PV is `Bound`;
- layer 5: the claim is bound to `block-asm-disk1-pv-vol1`;
- layer 6: `{"devicePath":"/dev/asm-disk1","name":"asmvol1"}`;
- layers 7-8: in both pods, `/dev/asm-disk1` with the same major:minor as that worker's disk in layers 2-3, owned `grid:asmadmin 660`;
- layer 9: ASM lists `/dev/asm-disk1` as a member of `DATA`.

Compared with udev rules on bare metal

On a bare-metal RAC node, a udev rule such as this does three jobs:

```text
KERNEL=="sd*", ENV{ID_SERIAL}=="<serial of the LUN>", SYMLINK+="oracleasm/disk1", OWNER="grid",
GROUP="asmadmin", MODE="0660"
```

JOB	BARE METAL	THIS LAB
Find the right disk whatever the kernel calls it	the rule matches the serial or WWID	udev's **default** rules create `/dev/disk/by-id/...` from the serial; the PV points at it (layers 3 and 4)
Give it a fixed name ASM can discover	`SYMLINK+="oracleasm/disk1"`	`devicePath: /dev/asm-disk1` in the pod spec (layer 6)

JOB	BARE METAL	THIS LAB
Owner `grid:asmadmin`, mode 660	`OWNER`, `GROUP`, `MODE`	`setupContainerEnv.sh` at every pod start (layer 8)

Why a udev rule on the workers wouldn't be enough:

- **The pod gets its own device node.** CRI-O creates `/dev/asm-disk1` inside the container (layer 7). Ownership set on the worker's `/dev/sdd` doesn't carry over.
- **No udev runs inside the pod**, so a rule in the image would never fire.
- **The name the database sees comes from the pod spec**, not from the host. `/dev/asm-disk1` is the same in both pods whatever the disk is called on each worker.

ASMLib and ASM Filter Driver are the other bare-metal options. Both need kernel modules on the host and their own disk labels; neither is used in this lab, which follows the doc's approach of plain block devices.

For a demo audience: the stable identity is set in two places, `/dev/disk/by-id` on the host (referenced by the PersistentVolume) and the fixed `devicePath` in the pod. Ownership and permissions are applied by the image's start-up script, because the container gets its own device node and runs no udev.

What goes wrong at each layer

LAYER	WHAT GOES WRONG	WHAT YOU SEE	PREVENTION OR FIX
1 VirtualBox	Disk not marked shareable, or attached to one worker only	VirtualBox refuses to start the second worker, or the second pod never sees the other's writes	`--type shareable` and `--mtype shareable` on both workers; guide 3.3's check covers both disks
1 VirtualBox	Snapshot of a worker	VirtualBox refuses	Don't snapshot the workers; snapshot the golden VM instead
2 Kernel	Formatting by `sdX` name	A shared ASM disk gets formatted (issue #7, caught in time)	Data disks chosen by size (guide 4.3/4.4, steps d and e)
3 udev	PV points at a name that doesn't exist on a worker	Pod stuck in `ContainerCreating`, with an event saying the path doesn't exist	Guide 7.2 checks both workers see the same by-id names
4 PV	Placeholder or wrong ID in the PV	Same as above, or, worse, a valid name of the wrong disk	Guide 7.2/7.3 fill the IDs in automatically (issue #14)
4 PV	`nodeAffinity` lists one worker only	The second RAC pod stays `Pending`	List both workers

LAYER	WHAT GOES WRONG	WHAT YOU SEE	PREVENTION OR FIX
5 PVC	No label selector	Claim 1 may bind to disk 2. ASM would still find its disks by their headers, but `/dev/asm-disk1` would no longer be asm1.vdi, which makes tracing confusing	`selector` on `asm_vol`
6 Pod spec	`volumeMounts` instead of `volumeDevices`, or `volumeMode` missing	Pod fails to start: the volume mode doesn't match	`volumeMode: Block` in PV and PVC, `volumeDevices` in the pod
7 Runtime	Device not allowed in the cgroup (e.g. a hand-made `mknod` in the pod)	"Operation not permitted" opening the device, even as root	Only use devices listed in `volumeDevices`
8 Ownership	Script didn't run, or the image lacks it	Devices `root:root` (or `root:disk`); ASM can't open them; the Grid install can't find candidate disks	`setupContainerEnv.sh` in the image; guide 10.5 checks `grid:asmadmin 660`
9 ASM	Disks still carry an old ASM header	The Grid install says the disks are already in use	Delete and recreate the VDIs (guide 0.0 and 3.3)

In production

The lab keeps the chain as short as possible. In production the same layers exist, with sturdier pieces:

- **Layer 1:** SAN LUNs (Fibre Channel or iSCSI) or NVMe-oF namespaces, presented to every worker that can run a RAC pod.
- **Layers 2-3:** multipath. Each LUN reaches the worker by several paths, and `multipathd` combines them into one device such as `/dev/mapper/<wwid>`, with a matching `/dev/disk/by-id/dm-uuid-mpath-<wwid>` link. The PV then points at the multipath device, never at a single path.
- **Layer 4:** either the same static local PVs, created per LUN, or a **CSI driver** from the storage vendor that supports raw block volumes with `ReadWriteMany`. CSI can create volumes on demand from a StorageClass. Check the driver supports shared block access before relying on it for RAC.
- **Layer 8:** the same idea, ownership applied in the container at start. Some teams use an init container or the runtime's device ownership settings instead of a script.
- **Layer 9:** usually NORMAL or HIGH redundancy across failure groups, or EXTERNAL when the array already provides RAID; several disk groups (data, recovery area) rather than one.

What doesn't change is the principle the lab demonstrates: the host finds the disk by a stable identity, Kubernetes maps it to a fixed name in the pod, and ASM and RAC, not Kubernetes, coordinate the shared access.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.